

Sequences : Lists and Tuples

Lec-17_18_19_20_21

Lists

- To create a list,

```
x = list()
```

```
y = []
```

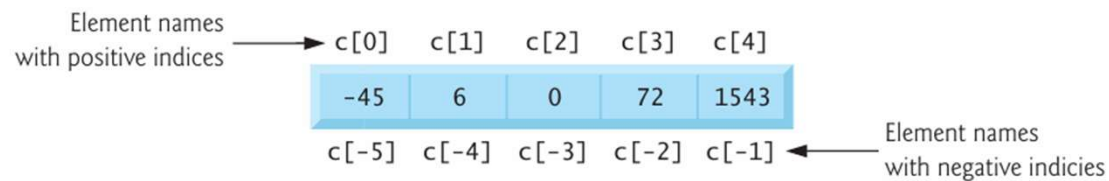
- Lists can store homogeneous/heterogeneous data, that is, values of the same data type. Consider the list `c`, which contains five integer elements.

```
c = [-45, 6, 0, 72, 1543]
```

```
d = ['Mary', 'Smith', 3.57, 2022]
```

- You reference a list element by writing the list's name followed by the element's index (that is, its position number) enclosed in square brackets (`[]`), known as the subscription operator).

Lists



- To access the first element, syntax is `list_name[position]`.
- To access the first element from the end use `list_name[-1]`.
- To find the number of elements in the list use the `len()`. Pass the list variable as an argument to `len()`.
- The position must be an int expression. Anything else will result in an error.
- Lists are mutable (The elements within the list can be modified).
- Attempting to access a non-existent index within a list results in an error.

Lists

- Ways to append to a list.
 - Using += : `a_list = [] ; a_list += [value]`
 - Using append : `b_list = [] ; b_list.append(value)`
- A string appended to list, breaks the string into single chars and stores them separately onto the list.
- `letters = [] ; letters += 'Goofiest' ; print(letters) # ['G', 'o', 'o', ... 't']`
- To access a list's indices and values use for and range.
 - `x = [1,2,3,4,5,6]`
 - For `i` in `range(len(x))`:
`print(f'i={i} : val={x[i]}')`
- Two lists can be concatenated using the + operator.
 - `l1, l2 = [1,2,3], [4,5,6]`
 - `l3 = l1 + l2`

Lists

- Lists can be compared using comparison operators.

```
In [29]: a = [1, 2, 3]
```

```
In [30]: b = [1, 2, 3]
```

```
In [31]: c = [1, 2, 3, 4]
```

```
In [32]: a == b # True: corresponding elements in both are equal
```

```
Out[32]: True
```

```
In [33]: a == c # False: a and c have different elements and lengths
```

```
Out[33]: False
```

```
In [34]: a < c # True: a has fewer elements than c
```

```
Out[34]: True
```

```
In [35]: c >= b # True: elements 0-2 are equal but c has more elements
```

```
Out[35]: True
```

Tuples

- Creating tuples:
 - `S_tuple = tuple(); T_tuple = ()` # However it is totally useless. It cannot be modified or used.
 - `S_tuple = 'abc', 1.0, 2` # a tuple with 3 values.
 - To create a tuple with single element : `t = (1)` # this will create an int not a tuple. The correct way is `t = (1,)`
- Tuples are immutable. `t = (1,); t += (2,3,4); print(t)`
- Concatenating tuple with another will create a new tuple then assigns a new reference to t.
- A tuple can be concatenated to a list. But vice versa is not allowed.

Tuples

- Tuples May Contain Mutable Objects.

```
In [23]: student_tuple = ('Amanda', 'Blue', [98, 75, 87])
```

Even though the tuple is immutable, its list element is mutable:

```
In [24]: student_tuple[2][1] = 85
```

```
In [25]: student_tuple
```

```
Out[25]: ('Amanda', 'Blue', [98, 85, 87])
```

In the *double-subscripted name* `student_tuple[2][1]`, Python views `student_tuple[2]` as the element of the tuple containing the list `[98, 75, 87]`, then uses `[1]` to access the list element containing 75. The assignment in snippet [24] replaces that grade with 85.

- You can unpack any sequence's elements by assigning the sequence to a comma-separated list of variables. A `ValueError` occurs if the number of variables to the left of the assignment symbol is not identical to the number of elements in the sequence on the right

Tuples

- Tuple Unpacking

```
In [1]: student_tuple = ('Amanda', [98, 85, 87])
```

```
In [2]: first_name, grades = student_tuple
```

```
In [3]: first_name  
Out[3]: 'Amanda'
```

```
In [4]: grades  
Out[4]: [98, 85, 87]
```

The following code unpacks a string, a list and a sequence produced by range:

```
In [5]: first, second = 'hi'
```

```
In [6]: print(f'{first} {second}')  
h i
```

You can swap two variables' values using sequence packing and unpacking:

```
In [11]: number1 = 99
```

```
In [12]: number2 = 22
```

```
In [13]: number1, number2 = (number2, number1)
```

```
In [14]: print(f'number1 = {number1}; number2 = {number2}')  
number1 = 22; number2 = 99
```

Accessing Indices and Values Safely with Built-in Function `enumerate`

- The mechanism for accessing an element's index and value is the built-in function `enumerate`. This function receives an iterable and creates an iterator that, for each element, returns a tuple containing the element's index and value. The following code uses the built-in function `list` to create a list containing `enumerate`'s results

Enumerate

- Example Usage

```
In [15]: colors = ['red', 'orange', 'yellow']
```

```
In [16]: list(enumerate(colors))
```

```
Out[16]: [(0, 'red'), (1, 'orange'), (2, 'yellow')]
```

Similarly the built-in function **tuple** creates a tuple from a sequence:

```
In [17]: tuple(enumerate(colors))
```

```
Out[17]: ((0, 'red'), (1, 'orange'), (2, 'yellow'))
```

The following for loop unpacks each tuple returned by `enumerate` into the variables `index` and `value` and displays them:

```
In [18]: for index, value in enumerate(colors):
```

```
...:     print(f'{index}: {value}')
```

```
...:
```

```
0: red
```

```
1: orange
```

```
2: yellow
```

Sequence Slicing

- You can slice sequences to create new sequences of the same type containing subsets of the original elements. Slice operations can modify mutable sequences—those that do not modify a sequence work identically for lists, tuples and strings.

Specifying a Slice with Starting and Ending Indices

Let's create a slice consisting of the elements at indices 2 through 5 of a list:

```
In [1]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]
```

```
In [2]: numbers[2:6]
```

```
Out[2]: [5, 7, 11, 13]
```

The slice *copies* elements from the *starting index* to the left of the colon (2) up to, but not including, the *ending index* to the right of the colon (6). The original list is not modified.

Sequence Slicing

Specifying a Slice with Only an Ending Index

If you omit the starting index, 0 is assumed. So, the slice `numbers[:6]` is equivalent to the slice `numbers[0:6]`:

```
In [3]: numbers[:6]
Out[3]: [2, 3, 5, 7, 11, 13]
```

```
In [4]: numbers[0:6]
Out[4]: [2, 3, 5, 7, 11, 13]
```

Specifying a Slice with Only a Starting Index

If you omit the ending index, Python assumes the sequence's length (8 here), so snippet `[5]`'s slice contains the elements of `numbers` at indices 6 and 7:

```
In [5]: numbers[6:]
Out[5]: [17, 19]
```

```
In [6]: numbers[6:len(numbers)]
Out[6]: [17, 19]
```

Sequence Slicing

Specifying a Slice with No Indices

Omitting both the start and end indices copies the entire sequence:

```
In [7]: numbers[:]
Out[7]: [2, 3, 5, 7, 11, 13, 17, 19]
```

Though slices create new objects, slices make *shallow* copies of the elements—that is, they copy the elements’ references but not the objects they point to. So, in the snippet above, the new list’s elements refer to the *same objects* as the original list’s elements, rather than to separate copies. In the “Array-Oriented Programming with NumPy” chapter, we’ll explain *deep* copying, which actually copies the referenced objects themselves, and we’ll point out when deep copying is preferred.

Slicing with Steps

The following code uses a *step* of 2 to create a slice with every other element of `numbers`:

```
In [8]: numbers[::2]
Out[8]: [2, 5, 11, 17]
```

We omitted the start and end indices, so 0 and `len(numbers)` are assumed, respectively.

Slicing with Negative Indices and Steps

You can use a negative step to select slices in *reverse* order. The following code concisely creates a new list in reverse order:

```
In [9]: numbers[::-1]
Out[9]: [19, 17, 13, 11, 7, 5, 3, 2]
```

This is equivalent to:

```
In [10]: numbers[-1:-9:-1]
Out[10]: [19, 17, 13, 11, 7, 5, 3, 2]
```

Sequence Slicing

- You can modify a list by assigning to a slice of it—the rest of the list is unchanged.

```
In [11]: numbers[0:3] = ['two', 'three', 'five']
```

```
In [12]: numbers
```

```
Out[12]: ['two', 'three', 'five', 7, 11, 13, 17, 19]
```

The following deletes only the first three elements of `numbers` by assigning an *empty* list to the three-element slice:

```
In [13]: numbers[0:3] = []
```

```
In [14]: numbers
```

```
Out[14]: [7, 11, 13, 17, 19]
```

The following assigns a list's elements to a slice of every other element of `numbers`:

```
In [15]: numbers = [2, 3, 5, 7, 11, 13, 17, 19]
```

```
In [16]: numbers[::2] = [100, 100, 100, 100]
```

```
In [17]: numbers
```

```
Out[17]: [100, 3, 100, 7, 100, 13, 100, 19]
```

```
In [18]: id(numbers)
```

```
Out[18]: 4434456648
```

Let's delete all the elements in `numbers`, leaving the *existing* list empty:

```
In [19]: numbers[:] = []
```

```
In [20]: numbers
```

```
Out[20]: []
```

```
In [21]: id(numbers)
```

```
Out[21]: 4434456648
```

Sequence Slicing

Deleting numbers' contents (snippet [19]) is different from assigning numbers a *new* empty list [] (snippet [22]). To prove this, we display numbers' identity after each operation. The identities are different, so they represent separate objects in memory:

```
In [22]: numbers = []
```

```
In [23]: numbers
```

```
Out[23]: []
```

```
In [24]: id(numbers)
```

```
Out[24]: 4406030920
```

When you assign a new object to a variable (as in snippet [21]), the original object will be garbage collected if no other variables refer to it.

Del Statement

5.6 del Statement

The **del statement** also can be used to remove elements from a list and to delete variables from the interactive session. You can remove the element at any valid index or the element(s) from any valid slice.

Deleting the Element at a Specific List Index

Let's create a list, then use `del` to remove its last element:

```
In [1]: numbers = list(range(0, 10))  
  
In [2]: numbers  
Out[2]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
In [3]: del numbers[-1]  
  
In [4]: numbers  
Out[4]: [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Deleting a Slice from a List

The following deletes the list's first two elements:

```
In [5]: del numbers[0:2]  
  
In [6]: numbers  
Out[6]: [2, 3, 4, 5, 6, 7, 8]
```

Del Statement

The following uses a step in the slice to delete every other element from the entire list:

```
In [7]: del numbers[::2]
```

```
In [8]: numbers
```

```
Out[8]: [3, 5, 7]
```

Deleting a Slice Representing the Entire List

The following code deletes all of the list's elements:

```
In [9]: del numbers[:]
```

```
In [10]: numbers
```

```
Out[10]: []
```

Deleting a Variable from the Current Session

The `del` statement can delete any variable. Let's delete `numbers` from the interactive session, then attempt to display the variable's value, causing a `NameError`:

```
In [11]: del numbers
```

```
In [12]: numbers
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-12-426f8401232b> in <module>()  
----> 1 numbers
```

```
NameError: name 'numbers' is not defined
```

Passing Lists to functions

Passing an Entire List to a Function

Consider the function `modify_elements`, which receives a reference to a list and multiplies each of the list's element values by 2:

```
In [1]: def modify_elements(items):
...:     """Multiplies all element values in items by 2."""
...:     for i in range(len(items)):
...:         items[i] *= 2
...:

In [2]: numbers = [10, 3, 7, 1, 9]

In [3]: modify_elements(numbers)

In [4]: numbers
Out[4]: [20, 6, 14, 2, 18]
```

Function `modify_elements`' `items` parameter receives a reference to the *original* list, so the statement in the loop's suite modifies each element in the original list object.

Passing a Tuple to a Function

When you pass a tuple to a function, attempting to modify the tuple's immutable elements results in a `TypeError`:

```
In [5]: numbers_tuple = (10, 20, 30)

In [6]: numbers_tuple
Out[6]: (10, 20, 30)

In [7]: modify_elements(numbers_tuple)
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-27-9339741cd595> in <module>()
----> 1 modify_elements(numbers_tuple)

<ipython-input-25-27acb8f8f44c> in modify_elements(items)
      2     """Multiplies all element values in items by 2."""
      3     for i in range(len(items)):
----> 4         items[i] *= 2
      5
      6

TypeError: 'tuple' object does not support item assignment
```

Sorting Lists

A common computing task called **sorting** enables you to arrange data either in ascending or descending order. Sorting is an intriguing problem that has attracted intense computer-science research efforts. It's studied in detail in data-structures and algorithms courses. We discuss sorting in more detail in the “Computer Science Thinking: Recursion, Searching, Sorting and Big O” chapter.

Sorting a List in Ascending Order

List method **sort** *modifies* a list to arrange its elements in ascending order:

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]
In [2]: numbers.sort()
In [3]: numbers
Out[3]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Sorting a List in Descending Order

To sort a list in descending order, call list method `sort` with the optional keyword argument **reverse** set to `True` (`False` is the default):

```
In [4]: numbers.sort(reverse=True)
In [5]: numbers
Out[5]: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Built-In Function `sorted`

Built-in function **sorted** *returns a new list* containing the sorted elements of its argument *sequence*—the original sequence is *unmodified*. The following code demonstrates function `sorted` for a list, a string and a tuple:

```
In [6]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]
In [7]: ascending_numbers = sorted(numbers)
```

Searching Sequences

- Searching is the process of locating a key.

List Method `index`

List method `index` takes as an argument a search key—the value to locate in the list—then searches through the list from index 0 and returns the index of the *first* element that matches the search key:

```
In [1]: numbers = [3, 7, 1, 4, 2, 8, 5, 6]
```

```
In [2]: numbers.index(5)  
Out[2]: 6
```

A `ValueError` occurs if the value you're searching for is not in the list.

The following code searches the updated list for the value 5 starting from index 7 and continuing through the end of the list:

```
In [5]: numbers.index(5, 7)  
Out[5]: 14
```

Searching Sequences

Specifying the Starting and Ending Indices of a Search

Specifying the starting and ending indices causes `index` to search from the starting index up to but not including the ending index location. The call to `index` in snippet [5]:

```
numbers.index(5, 7)
```

assumes the length of `numbers` as its optional third argument and is equivalent to:

```
numbers.index(5, 7, len(numbers))
```

The following looks for the value 7 in the range of elements with indices 0 through 3:

```
In [6]: numbers.index(7, 0, 4)
Out[6]: 1
```

```
In [7]: 1000 in numbers
Out[7]: False
```

```
In [8]: 5 in numbers
Out[8]: True
```

- Trying to find an index of an element that doesn't exist can result in an error. To avoid this we use the `in` and `not in` operators before using the `.index` method

So the recommended way to search is:

if key in lst:

```
print(lst.index(key))
```

Functions `any` and `all`

Built-In Functions `any` and `all`

Sometimes you simply need to know whether *any* item in an iterable is `True` or whether *all* the items are `True`. The built-in function `any` returns `True` if any item in its iterable argument is `True`. The built-in function `all` returns `True` if all items in its iterable argument are `True`. Recall that nonzero values are `True` and `0` is `False`. Non-empty iterable objects also evaluate to `True`, whereas any empty iterable evaluates to `False`. Functions `any` and `all` are additional examples of internal iteration in functional-style programming.

Other List Methods

Lists also have methods that add and remove elements. Consider the list `color_names`:

```
In [1]: color_names = ['orange', 'yellow', 'green']
```

Inserting an Element at a Specific List Index

Method **`insert`** adds a new item at a specified index. The following inserts 'red' at index 0:

```
In [2]: color_names.insert(0, 'red')  
  
In [3]: color_names  
Out[3]: ['red', 'orange', 'yellow', 'green']
```

Adding an Element to the End of a List

You can add a new item to the end of a list with method **`append`**:

```
In [4]: color_names.append('blue')  
  
In [5]: color_names  
Out[5]: ['red', 'orange', 'yellow', 'green', 'blue']
```

Adding All the Elements of a Sequence to the End of a List

Use list method **`extend`** to add all the elements of another sequence to the end of a list:

```
In [6]: color_names.extend(['indigo', 'violet'])  
  
In [7]: color_names  
Out[7]: ['red', 'orange', 'yellow', 'green', 'blue', 'indigo', 'violet']
```

This is the equivalent of using `+=`. The following code adds all the characters of a string then all the elements of a tuple to a list:

Difference between `append` and `extend`?

`.append()` adds its argument as a single element to the end of a list. The length of the list itself will increase by one.
`.extend()` iterates over its argument adding each element to the list, extending the list. The length of the list will increase by however many elements were in the iterable argument.

Other List Methods

Removing the First Occurrence of an Element in a List

Method **remove** deletes the first element with a specified value—a `ValueError` occurs if `remove`'s argument is not in the list:

```
In [17]: color_names.remove('green')

In [18]: color_names
Out[18]: ['red', 'orange', 'yellow', 'blue', 'indigo', 'violet']
```

Emptying a List

To delete all the elements in a list, call method **clear**:

```
In [19]: color_names.clear()

In [20]: color_names
Out[20]: []
```

This is the equivalent of the previously shown slice assignment

```
color_names[:] = []
```

Counting the Number of Occurrences of an Item

List method **count** searches for its argument and returns the number of times it is found:

```
In [21]: responses = [1, 2, 5, 4, 3, 5, 2, 1, 3, 3,
...:                  1, 4, 3, 3, 3, 2, 3, 3, 2, 2]
...:
```

```
In [22]: for i in range(1, 6):
...:     print(f'{i} appears {responses.count(i)} times in responses')
...:
1 appears 3 times in responses
2 appears 5 times in responses
3 appears 8 times in responses
4 appears 2 times in responses
5 appears 2 times in responses
```

Reversing a List's Elements

List method **reverse** reverses the contents of a list in place, rather than creating a reversed copy, as we did with a slice previously:

```
In [23]: color_names = ['red', 'orange', 'yellow', 'green', 'blue']

In [24]: color_names.reverse()

In [25]: color_names
Out[25]: ['blue', 'green', 'yellow', 'orange', 'red']
```

Other List methods

Copying a List

List method `copy` returns a *new* list containing a *shallow* copy of the original list:

```
In [26]: copied_list = color_names.copy()
```

```
In [27]: copied_list
```

```
Out[27]: ['blue', 'green', 'yellow', 'orange', 'red']
```

This is equivalent to the previously demonstrated slice operation:

```
copied_list = color_names[:]
```

Simulating Stack Data Structure with Lists

- Stack has 4 methods. Push, pop, peek, isEmpty
- Push can be implemented using `list_var.append()`
- Pop can be implemented using `list_var.pop()`
- Peek can be implemented by returning or printing the top most or the last element of the list. `list_var[-1]`
- isEmpty can be implemented by returning true if `len(list_var) == 0`

List Comprehension

- A **list comprehension** in Python is a tool for creating lists by iterating over an iterable and optionally applying a condition.
- You should use **list comprehensions** instead of loops when you want concise, readable code that performs transformations or filtering.
- You add **conditional logic** to a list comprehension by including an if statement within the comprehension.
- A **list comprehension** can be faster than a for loop because it's optimized for performance by Python's internal mechanisms.
- Syntax:
 - `new_list = [expression for member in iterable if conditional]`
 - **expression** is the member itself, a call to a method, or any other valid expression that returns a value.
 - **member** is the object or value in the list or iterable.
 - **iterable** is a list, [set](#), sequence, [generator](#), or any other object that can return its elements one at a time.

List Comprehension

- WAP to generate a list of numbers from [1,n)

P1 : Traditional Method

```
l1 = []
```

```
for i in range(1,n):
```

```
    l1.append(i)
```

P2 : LC method

```
L2 = [item for item in range(1,n)]
```

List Comprehension

- WALC to generate a list of even numbers between a user defined range.

`X = [num for num in range(r1, r2) if num % 2 == 0]`

- WALC to generate a list of odd numbers between a user defined range.

List Comprehension

- **Create a list of even numbers squared and odd numbers cubed from 1 to 10**

```
def square_or_cube(x):  
    if x % 2 == 0:  
        return x * x  
    else:  
        return x * x * x  
print([square_or_cube(item) for item in range(1,11)])
```

List Comprehension

- Generate a list of characters from a string
- Create a list of lengths of words in a sentence
- Generate a list of tuples containing a number and its square
- Create a list of lowercase letters
- Generate a list of common multiples of 3 and 5 up to 100

Generator Expressions

- A generator expression is similar to a list comprehension, but creates an iterable generator object that produces values on demand. This is known as lazy evaluation. List comprehensions use greedy evaluation—they create lists immediately when you execute them. For large numbers of items, creating a list can take substantial memory and time. So, generator expressions can reduce your program's memory consumption and improve performance if the whole list is not needed at once.
- Generator expressions have the same capabilities as list comprehensions, but you define them in parentheses instead of square brackets.

Filter, Map and Reduce

- Filter, map and reduce are higher order functions that take a function as an argument.

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]
```

```
In [2]: def is_odd(x):  
...:     """Returns True only if x is odd."""  
...:     return x % 2 != 0  
...:
```

```
In [3]: list(filter(is_odd, numbers))
```

```
Out[3]: [3, 7, 1, 9, 5]
```

Filter, Map and Reduce

- Like data, Python functions are objects that you can assign to variables, pass to other functions and return from functions. **Functions that receive other functions as arguments are a functional-style capability called higher-order functions.** For example, filter's first argument must be a function that receives one argument and returns True if the value should be included in the result. The function `is_odd` returns True if its argument is odd. The filter function calls `is_odd` once for each value in its second argument's iterable (numbers). Higher-order functions may also return a function as a result. **Function filter returns an iterator, so filter's results are not produced until you iterate through them. This is another example of lazy evaluation.**

Filter, Map and Reduce

- For simple functions like `is_odd` that return only a single expression's value, you can use a lambda expression (or simply a lambda) to define the function inline where it's needed— typically as it's passed to another function

```
list(filter(lambda x: x % 2 != 0, numbers))  
[3, 7, 1, 9, 5]
```

- A lambda expression is an anonymous function—that is, a function without a name.
- A lambda begins with the lambda keyword followed by a comma-separated parameter list, a colon (:) and an expression.

```
def function_name(parameter_list):  
    lambda parameter_list: expression  
    return expression
```

Filter, Map and Reduce

Let's use built-in function `map` with a lambda to square each value in `numbers`:

```
In [6]: numbers
```

```
Out[6]: [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]
```

```
In [7]: list(map(lambda x: x ** 2, numbers))
```

```
Out[7]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Function `map`'s first argument is a function that receives one value and returns a new value—in this case, a lambda that squares its argument. The second argument is an iterable of values to map. Function `map` uses lazy evaluation. So, we pass to the `list` function the iterator that `map` returns. This enables us to iterate through and create a list of the mapped values. Here's an equivalent list comprehension:

```
In [8]: [item ** 2 for item in numbers]
```

```
Out[8]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Filter, Map and Reduce

- Find the output of the code given numbers = [10,3,7,1,9,4,2,8,5,6]:

```
list(map(lambda x: x ** 2,  
        filter(lambda x: x % 2 != 0, numbers)))
```

- Write the equivalent list comprehension for the above.

Other Sequence Processing Functions

- Given a list of strings. Find the maximum string alphabetically.
- `colors = ['Red', 'orange', 'Yellow', 'green', 'Blue']`
- Expected answer : 'Yellow'
- O/P : 'orange'
- One way to solve is to convert all the strings into lowercase and then to compare it.
- It can be solved in a one liner beautifully using 'key' and lambdas.
- One Liner Solution = `max(colors, key=lambda s : s.lower())`

Other Sequence Processing Functions

Iterating Backward Through a Sequence

Built-in function **reversed** returns an iterator that enables you to iterate over a sequence's values backward. The following list comprehension creates a new list containing the squares of numbers' values in reverse order:

```
In [7]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]
```

```
In [7]: reversed_numbers = [item for item in reversed(numbers)]
```

```
In [8]: reversed_numbers
```

```
Out[8]: [36, 25, 64, 4, 16, 81, 1, 49, 9, 100]
```

Other Sequence Processing Functions

Combining Iterables into Tuples of Corresponding Elements

Built-in function **zip** enables you to iterate over *multiple* iterables of data at the *same* time. The function receives as arguments any number of iterables and returns an iterator that produces tuples containing the elements at the same index in each. For example, snippet [11]'s call to **zip** produces the tuples ('Bob', 3.5), ('Sue', 4.0) and ('Amanda', 3.75) consisting of the elements at index 0, 1 and 2 of each list, respectively:

```
In [9]: names = ['Bob', 'Sue', 'Amanda']

In [10]: grade_point_averages = [3.5, 4.0, 3.75]

In [11]: for name, gpa in zip(names, grade_point_averages):
...:     print(f'Name={name}; GPA={gpa}')
...:
Name=Bob; GPA=3.5
Name=Sue; GPA=4.0
Name=Amanda; GPA=3.75
```

We unpack each tuple into **name** and **gpa** and display them. Function **zip**'s shortest argument determines the number of tuples produced. Here both have the same length.

2D Lists

- Lists can contain other lists as elements. A typical use of such nested (or multidimensional) lists is to represent tables of values consisting of information arranged in rows and columns. To identify a particular table element, we specify two indices—by convention, the first identifies the element's row, the second the element's column.
- Lists that require two indices to identify an element are called two-dimensional lists (or double-indexed lists or double-subscripted lists). Multidimensional lists can have more than two indices. Here, we introduce two-dimensional lists.
- To create a 2D list use the syntax:
 - `a = [[77, 68, 86, 73], [96, 87, 89, 81], [70, 90, 86, 81]]`
 - `A = []`
 - `for x in range(4):`
 - `Z = [1]*5`
 - `A.append(Z)`

2D Lists

Illustrating a Two-Dimensional List

The diagram below shows the list `a`, with its rows and columns of exam grade values:

	Column 0	Column 1	Column 2	Column 3
Row 0	77	68	86	73
Row 1	96	87	89	81
Row 2	70	90	86	81

Identifying the Elements in a Two-Dimensional List

The following diagram shows the names of list `a`'s elements:

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Diagram illustrating the naming convention for elements in a 2D list `a`:

- Column index: Points to the column index (1) in the element name `a[2][1]`.
- Row index: Points to the row index (2) in the element name `a[2][1]`.
- List name: Points to the list name `a` in the element name `a[2][1]`.

Questions

- WAP to create a 2D List of 4 x 5, filled with 0s using LC.
- WALC to that generates 600 random numbers from [1-6].